You can have multiple templates with the same name but with different extensions (for example, `.html.erb`, `.xml.builder`, and `.js.erb`). If you don't specify an extension in a render request, Rails assumes `html.erb`.

Rendering Templates

A *template* is a file that defines the content of a response for our application. Rails supports three template formats out of the box: *erb*, which is embedded Ruby code (typically with HTML); *builder*, a more programmatic way of constructing XML content; and *RJS*, which generates JavaScript. We'll talk about the contents of these files starting in *Using Templates*.

By convention, the template for *action* of *controller* will be in the file `app/views/controller/action.type.xxx` (where *type* is the file type, such as `html`, `atom`, or `js`; and *xxx* is one of `erb`, `builder`, or `scss`). The `app/views` part of the name is the default. You can override this for an entire application by setting this:

`ActionController.prepend_view_path` *`dir_path`*

The `render` method is the heart of all rendering in Rails. It takes a hash of options that tell it what to render and how to render it.

It's tempting to write code in our controllers that looks like this:

```
# DO NOT DO THIS
def update
  @user = User.find(params[:id])
  if @user.update(user_params)
    render action: show
  end
  render template: "fix_user_errors"
end
```

It seems somehow natural that the act of calling `render` (and `redirect_to`) should somehow terminate the processing of an action. This isn't the case. The previous code will generate an error (because `render` is called twice) in the case where `update` succeeds.